hexens x ROYCO

Security Review Report for Royco

March 2026



Table of Contents

1. About Hexens
2. Executive summary
3. Security Review Details
    - Security Review Lead
    - Scope
    - Changelog
4. Severity Structure
    - Severity characteristics
    - Issue symbolic codes
5. Findings Summary
6. Weaknesses
    - Missing LLTV Enforcement on ST Deposit Enables JT Capital Extraction
    - Attacker can extract profit from the ST self-liquidation bonus
    - RWA Kernels Should Use Soulbound Tokens for Beneficial Ownership Tracking
    - Permission check is incorrectly skipped when minting in MaplePoolV2_ST_JT_ExitSharePriceToChainlinkOracle_Kernel
    - Lack of Target LTV Cap in Self-Liquidation Bonus Results
    - Tranche vault could be exploited for infinite-transfer or fee-on-transfer tokens
    - Redundant blacklist check in balance update hook
    - Missing event for market state to perpetual state
    - Redundant variable jtClaimOnSelfRawNAV in _applySeniorTrancheSelfLiquidationBonus
    - Redundant capping of jtYieldShareWAD in _computeCurrentJtYieldShare

1. About Hexens

Hexens is a pioneering cybersecurity firm dedicated to establishing robust security standards for Web3 infrastructure, driving secure mass adoption through innovative protection technology and frameworks. As an industry elite experts in blockchain security, we deliver comprehensive audit solutions across specialized domains, including infrastructure security, Zero Knowledge Proof, novel cryptography, DeFi protocols, and NFTs.

Our methodology combines industry-standard security practices combined with unique methodology of two teams per audit, continuously advancing the field of Web3 security. This innovative approach has earned us recognition from industry leaders.

Since our founding in 2021, we have built an exceptional portfolio of enterprise clients, including major blockchain ecosystems and Web3 platforms.

2. Executive Summary

This audit reviews an update to Royco, a perpetual risk-tranching protocol. The update focuses on a significant simplification of tranches and kernels, along with the introduction of a self-liquidation bonus mechanism. It also includes several changes to align the protocol with the RWA integration framework.

Our review was conducted over a one-week period and involved a comprehensive analysis of all relevant code changes.

During the assessment, we identified one high-severity issue related to missing LLTV enforcement on ST deposits, which could allow a malicious Senior Tranche supplier to extract profit through the liquidation bonus mechanism. In addition, we discovered two medium-severity issues, one low-severity issue, and five informational findings.

All identified issues were either remediated or acknowledged by the development team and subsequently verified by our auditors.

Following the remediation phase, we conclude that the protocol's overall security posture and code quality have improved as a result of this audit.

3. Security Review Details

Review Led by

Trung Dinh, Lead Security Researcher

Scope

The analyzed resources are located on:

🔗 https://github.com/roycoprotocol/royco-dawn/tree/4fdb810b1ded60134432d68814c639be90eb5eaa-

🔗 PR 59: https://github.com/roycoprotocol/royco-dawn/commit/580fd258665fdeb0af800b9a975242016d0dd8de

The issues described in this report were fixed in the following PR:

🔗 https://github.com/roycoprotocol/royco-dawn/pull/54

📌 **Commit:** `eac4da7d644ea211e53fc6ca96059bfefe6c4fcd`

Changelog

2 March 2025	Audit start
10 March 2026	Initial report
11 March 2026	Revision received
13 March 2026	Final report

4. Severity Structure

The vulnerability severity is calculated based on two components:

1. Impact of the vulnerability
2. Probability of the vulnerability

Impact	Probability			
	Rare	Unlikely	Likely	Very likely
Low	Low	Low	Medium	Medium
Medium	Low	Medium	Medium	High
High	Medium	Medium	High	Critical
Critical	Medium	High	Critical	Critical

- **Severity Characteristics**

Smart contract vulnerabilities can range in severity and impact, and it's important to understand their level of severity in order to prioritize their resolution. Here are the different types of severity levels of smart contract vulnerabilities:

Critical

Vulnerabilities that are highly likely to be exploited and can lead to catastrophic outcomes, such as total loss of protocol funds, unauthorized governance control, or permanent disruption of contract functionality.

High

Vulnerabilities that are likely to be exploited and can cause significant financial losses or severe operational disruptions, such as partial fund theft or temporary asset freezing.

Medium	Vulnerabilities that may be exploited under specific conditions and result in moderate harm, such as operational disruptions or limited financial impact without direct profit to the attacker.
Low	Vulnerabilities with low exploitation likelihood or minimal impact, affecting usability or efficiency but posing no significant security risk.
Informational	Issues that do not pose an immediate security risk but are relevant to best practices, code quality, or potential optimizations.

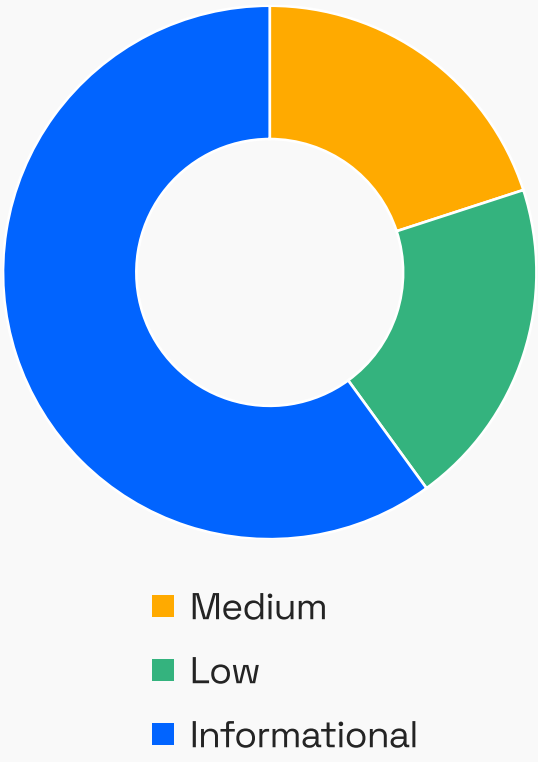
- **Issue Symbolic Codes**

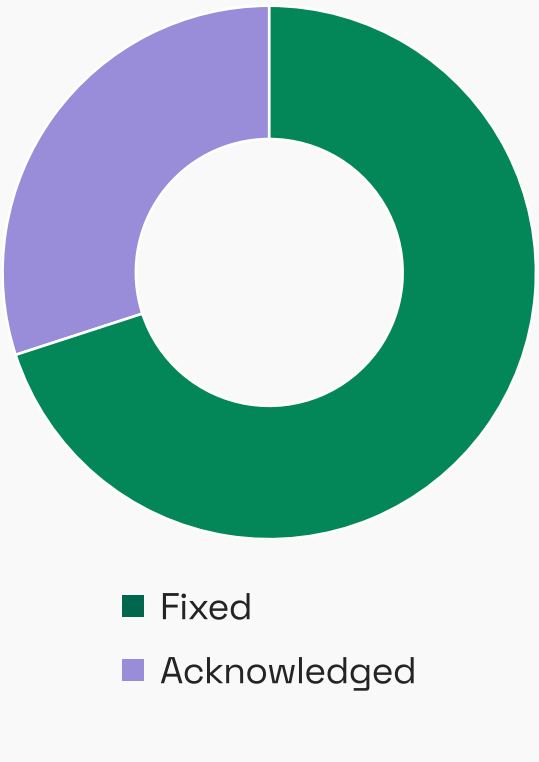
Each identified and validated issue is assigned a unique symbolic code during the security research stage.

Due to the structure of the vulnerability reporting flow, some rejected issues may be missing.

5. Findings Summary

Severity	Number of findings
Critical	0
High	0
Medium	2
Low	2
Informational	6
Total:	**10**





6. Weaknesses

This section contains the list of discovered weaknesses.

ROYCO3-5 | Missing LLTV Enforcement on ST Deposit Enables JT Capital Extraction

Fixed

Severity:	Probability:	Impact:
Medium	Likely	Medium

Path:

src/accountant/RoycoAccountant.sol::stDeposit()

Description:

A logical flaw in the Senior Tranche deposit accounting allows users to force the market into a self-liquidation state and extract capital from the Junior Tranche. The vulnerability occurs because the **stDeposit** post-operation synchronization only verifies the market coverage requirement (utilization <= 1) and fails to enforce the Liquidation Loan-to-Value (LLTV) threshold (ltv < lltv).

```
    function postOpSyncTrancheAccountingAndEnforceCoverage(
        Operation _op,
        NAV_UNIT _stRawNAV,
        NAV_UNIT _jtRawNAV
    )
        external
        override(IRoycoAccountant)
        returns (SyncedAccountingState memory state)
    {
        state = postOpSyncTrancheAccounting(_op, _stRawNAV, _jtRawNAV, ZERO_NAV_UNITS);
        require(_isCoverageRequirementSatisfied(state.utilizationWAD),
COVERAGE_REQUIREMENT_UNSATISFIED());
    }
```

When the market experiences a downturn and JT capital has absorbed the ST losses, **stImpermanentLoss** remains zero. Under these conditions, an attacker can execute a large ST deposit. Because the coverage calculation uses the deflated raw NAV of the assets while LTV uses the sustained effective NAV, this large deposit maintains the coverage requirement but

pushes the market's LTV above the LLTV threshold.

If the attacker immediately redeems their newly minted ST shares, the pre-operation sync detects the LLTV breach, transitioning the market into a **PERPETUAL** state and activating the self-liquidation bonus.

```
        if (
            fixedTermDurationSeconds == 0 || (initialMarketState == MarketState.FIXED_TERM &&
fixedTermEndTimestamp <= block.timestamp) || ltvWAD >= lltvWAD
                || stImpermanentLoss != ZERO_NAV_UNITS
        ) {
            jtImpermanentLossErased = jtImpermanentLoss;
            jtImpermanentLoss = ZERO_NAV_UNITS;
            resultingMarketState = MarketState.PERPETUAL;
            fixedTermEndTimestamp = 0;
        } else if (jtImpermanentLoss <= $.stNAVDustTolerance) {
        ...
        }
```

The attacker receives this bonus subsidized directly by the JT effective NAV.

```
    function stRedeem(
        uint256 _shares,
        address _receiver,
        bool _bypassRedemptionRestrictions
    )
        external
        virtual
        override(IRoycoKernel)
        whenNotPaused
        onlySeniorTranche
        nonReentrant
        withQuoterCache
        returns (AssetClaims memory userAssetClaims)
    {
        SyncedAccountingState memory state;
        uint256 totalTrancheShares;
        (state, userAssetClaims, totalTrancheShares) = _preOpSyncTrancheAccounting(TrancheType.SENIOR);
        require(_bypassRedemptionRestrictions || state.marketState == MarketState.PERPETUAL,
ST_REDEEM_DISABLED_IN_FIXED_TERM_STATE());

        userAssetClaims = UtilsLib.scaleAssetClaims(userAssetClaims, _shares, totalTrancheShares);
```

```
        NAV_UNIT stSelfLiquidationBonusNAV;
        (userAssetClaims, stSelfLiquidationBonusNAV) = _applySeniorTrancheSelfLiquidationBonus(state,
    userAssetClaims);

        _withdrawAssets(userAssetClaims, _receiver);

        _postOpSyncTrancheAccounting(Operation.ST_REDEEM, stSelfLiquidationBonusNAV);
    }
```

The financial impact of this extraction is bounded by the bonus rate and the available JT capital, structured as: Profit_NAV ≈ min(depositNAV * bonusWAD, jtEffectiveNAV)

Remediation:

Add LLTV enforcement to stDeposit in addition to coverage checks.

1. Block deposits when already breached
   Revert if pre-deposit state is already at/above LLTV:
   - **preState.ltvWAD >= preState.lltvWAD**
2. Enforce LLTV headroom on requested deposit
   Compute the maximum LLTV-safe ST deposit and revert if requested deposit exceeds it:
   - **maxByLLTV = floor((lltv * jtEffectiveNAV) / (1 - lltv)) - stEffectiveNAV**
   - reject when **requestedDepositNAV > maxByLLTV**
3. Keep a final post-op invariant check
   After post-op sync, revert if:
   - **postState.ltvWAD >= postState.lltvWAD**

Also align quoting/capacity paths (**stMaxDeposit** / previews) with this LLTV rule to prevent quote-vs-execution mismatch.

ROYCO3-10 | Attacker can extract profit from the ST self-liquidation bonus

Fixed

Severity:	Probability:	Impact:
Medium	Likely	Medium

Path:

src/kernels/base/RoycoKernel.sol::_applySeniorTrancheSelfLiquidationBonus()#L807-L837

Description:

When the LLTV threshold is breached, the kernel incentivizes ST redemptions by providing a self-liquidation bonus sourced from JT.

However, this mechanism raises a concern: an attacker could deposit assets into ST and immediately redeem them to receive the bonus funded by JT. As a result, the attacker may extract profit from the liquidation bonus without incurring meaningful cost.

The main barrier for the attacker is the coverage requirement, which limits the maximum ST deposit to the coverage supported by the current JT effective NAV. Each time the attacker performs this strategy, the JT effective NAV decreases because of the bonus, which reduces the amount that can be deposited into ST and therefore decreases the amount that can be extracted over time.

```
function _applySeniorTrancheSelfLiquidationBonus(
    SyncedAccountingState memory _state,
    AssetClaims memory _stUserClaims
)
    internal
    view
    virtual
    returns (AssetClaims memory stUserClaimsWithBonus, NAV_UNIT stSelfLiquidationBonusNAV)
{
    // If the LLTV has not been breached, there is no ST self-liquidation bonus remitted
    if (_state.ltvWAD < _state.lltvWAD) return (_stUserClaims, ZERO_NAV_UNITS);


    // Compute the desired ST redemption based on the configured ST self-liquidation bonus
    NAV_UNIT desiredBonusNAV =
_stUserClaims.nav.mulDiv(_getRoycoKernelStorage().stSelfLiquidationBonusWAD, WAD,
Math.Rounding.Floor);
    // Clamp the actual bonus by the remaining JT controlled NAV
    stSelfLiquidationBonusNAV = UnitsMathLib.min(desiredBonusNAV, _state.jtEffectiveNAV);
```

```
    // Preemptively return if there is no remaining bonus capital to remit
    if (stSelfLiquidationBonusNAV == ZERO_NAV_UNITS) return (_stUserClaims, ZERO_NAV_UNITS);


    // Decompose the NAV claims for the Junior Tranche to get the NAV claims for sourcing the bonus
    (,, NAV_UNIT jtClaimOnSTRawNAV, NAV_UNIT jtClaimOnSelfRawNAV) = _decomposeNAVClaims(_state);
    // Compute the bonus NAV sourced from JT's claims on each tranche's NAV: prioritize ST assets over JT assets for sourcing
    // stSelfLiquidationBonusNAV <= (jtClaimOnSTRawNAV + jtClaimOnSelfRawNAV) since it was bounded by JT effective NAV already
    NAV_UNIT bonusFromJTClaimOnSTRawNAV = UnitsMathLib.min(stSelfLiquidationBonusNAV, jtClaimOnSTRawNAV);
    NAV_UNIT bonusFromJTClaimOnSelfRawNAV = (stSelfLiquidationBonusNAV - bonusFromJTClaimOnSTRawNAV);


    // Apply the derived bonus to the user's asset claims
    stUserClaimsWithBonus.stAssets = _stUserClaims.stAssets + stConvertNAVUnitsToTrancheUnits(bonusFromJTClaimOnSTRawNAV);
    stUserClaimsWithBonus.jtAssets = _stUserClaims.jtAssets + jtConvertNAVUnitsToTrancheUnits(bonusFromJTClaimOnSelfRawNAV);
    stUserClaimsWithBonus.nav = _stUserClaims.nav + stSelfLiquidationBonusNAV;
}
```

Remediation:

Consider blocking the ST deposit when the LLTV is breached.

ROYCO3-11 | RWA Kernels Should Use Soulbound Tokens for Beneficial Ownership Tracking

Fixed

Severity:	Probability:	Impact:
Low	Rare	Medium

Description:

To be compliant with RWA requirements, the protocol must ensure that beneficial ownership of pool shares can be accurately tracked and tied to KYC-verified entities.

The RWA documentation states:

Protocol adaptation: To track beneficial ownership of the pool, receipt tokens can be used to represent pro-rata ownership. However, this tracking is only meaningful if each receipt token holder can be directly linked to a KYC-verified person or entity. While one possible approach would be to allow receipt tokens to follow the same whitelist as the protocol itself, this raises regulatory concerns—particularly around whether peer-to-peer transactions should be considered, and the potential complications if tokens are also used in another whitelisted pool. To mitigate these risks, the initial approach will be to make receipt or representation tokens non-transferable by users. This ensures ownership remains confined to allowlisted, KYC-verified wallets and preserves the issuer's ability to maintain accurate UBO records

However, currently it allows receipt tokens to be transferred between whitelisted addresses. Although these transfers remain within an allowlist, this behavior does not follow the stated compliance approach of non-transferable representation tokens.

Allowing peer-to-peer transfers introduces potential complications for beneficial ownership tracking and UBO records.

As an example, the RWA token from Aave does not allow transfer between peers.

- Tokenization

Remediation:

Consider overriding the token transfer functionality for RWA-related kernels to transfers between users. Receipt tokens should be non-transferable (soulbound) and only minted or burned by the protocol.

ROYCO3-13 | Permission check is incorrectly skipped when minting in MaplePoolV2_ST_JT_ExitSharePriceToChainlinkOracle_Kernel

Fixed

Severity:	Probability:	Impact:
Low	Unlikely	Low

Path:

67092a0a040f477a2639b7b8fdda69e9c3323430/src/kernels/MaplePoolV2_ST_JT_ExitSharePriceToChainlinkOracle_Kernel.sol#L72

Description:

In the contract **MaplePoolV2_ST_JT_ExitSharePriceToChainlinkOracle_Kernel**, the function **_preTrancheBalanceUpdate()** skips the permission check of MaplePoolV2 during share minting or burning.

```
function _preTrancheBalanceUpdate(address _caller, address _from, address _to, uint256 _amount) internal view override(RoycoKernel) {
    // Preemptively return if this is a mint or redeem since the Maple pool checks permissions on minting/redeeming the underlying
    if (_from == address(0) || _to == address(0)) return;

    ...
}
```

However, skipping this check may allow an address, who does not have permission to hold MaplePoolV2 tokens, to hold tranche share. This issue arises from **RoycoVaultTranche.deposit()**, which allows the depositor (**msg.sender**) to specify an arbitrary **receiver** address for the tranche shares.

```
function deposit(TRANCHE_UNIT _assets, address _receiver) public virtual override whenNotPaused restricted returns (uint256 shares) {
    ...

    _mint(_receiver, shares);

    ...
}
```

As a result, even if the depositor (**msg.sender**) has permission to hold MaplePoolV2 tokens, the specified **receiver** may not. Because **_mint()** bypasses the Maple permission check, an address that is not on the allowed list receiver may still **receive** and hold the tranche shares.

The similar issue happens to the burning process.

Remediation:

For the minting process, it can be considered equivalent to a transfer from **msg.sender** to the **receiver**.

For the burning process, the situation is more complex because it effectively represents a transfer from the **owner** to the **receiver**. However, the **receiver** is not included in the inputs of **RoycoKernel.preTrancheBalanceUpdateHook()**.

ROYCO3-12 | Lack of Target LTV Cap in Self-Liquidation Bonus Results

Acknowledged

Severity:	Probability:	Impact:
Informational	Likely	Informational

Path:

/src/kernels/base/RoycoKernel.sol

Description:

The function **_applySeniorTrancheSelfLiquidationBonus** gives a bonus to Senior Tranche (ST) users who withdraw while the market is "breached" (LTV > LLTV).

The flaw is that the bonus is applied to the entire withdrawal amount as long as the market is breached at the start of the transaction. There is no "target" LTV where the bonus stops.

This means a single large withdrawal can receive a bonus on its full amount, even for the portion that surpasses the healthy/breach threshold.

As a result the ST ends up extracting more incentives from the JT than suppose to.

Example:
The market is at 81% LTV (Breach is after 80%).

- Scenario A (One Big Withdrawal): A user withdraws a large amount. This withdrawal causes the LTV from 81% to drop all the way down to 30%. Because the market was breached at the start, the user gets a 10% bonus on the entire amount.
  - Result: The Junior Tranche pays a 1e20 incentive.
- Scenario B (10 Small withdrawals): Ten withdrawals of the same total amount as scenario A. The first withdrawal drops the LTV to 79%.
  - Result: Only the first withdrawal gets a bonus. The other 9 withdrawals get 0 bonus because the market is now "healthy." The JT only paid 1e19 in incentives (10% of scenario A).

Remediation:

The bonus should only apply to the portion of the withdrawal that is needed to bring the market back to the target LLTV.

Commentary from the client:

“It is a mechanism design property.”

ROYCO3-3 | Tranche vault could be exploited for infinite-transfer or fee-on-transfer tokens

Acknowledged

Severity:	Probability:	Impact:
Informational	Rare	Informational

Path:

src/tranches/base/RoycoVaultTranche.sol:deposit#L66-L89

Description:

The RoycoVaultTranche relies on the underlying kernel to handle NAV calculations during a deposit or redeem into the vault.

The **deposit** function handles the ERC20 transfer of the underlying token using **safeTransferFrom** and consequently calls to the kernel with either **stDeposit** or **jtDeposit** using the given **_assets** parameter, no matter how much was actually transferred during the ERC20.transferFrom.

Furthermore, in the kernel deposit function, it will simply increase the stored balance directly with the parameter value:

```
function _stDepositAssets(TRANCHE_UNIT _stAssets) internal virtual {
    // Credit the deposited assets to the senior tranche
    RoycoKernelState storage $ = _getRoycoKernelStorage();
    $.stOwnedYieldBearingAssets = $.stOwnedYieldBearingAssets + _stAssets;
}

function _jtDepositAssets(TRANCHE_UNIT _jtAssets) internal virtual {
    // Credit the deposited assets to the junior tranche
    RoycoKernelState storage $ = _getRoycoKernelStorage();
    $.jtOwnedYieldBearingAssets = $.jtOwnedYieldBearingAssets + _jtAssets;
}
```

This approach is incompatible and even exploitable when used with either infinite-transfer or fee-on-transfer tokens.

The infinite-transfer token is an ERC20 token that uses the **type(uint256).max** as a special magic value that simply means full balance. A widely used example of such a token is the Compound V3 deposit share token for a pool, such as cUSDCv3. When calling transferFrom(msg.sender, address(this), value) and the user specifies type(uint256).max as value, then the transfer will succeed with just the balance of msg.sender.

Code example of Compound V3: comet/contracts/Comet.sol at ed6ebcd84ac00906e8e725716891d482f4bef8b9 · compound-finance/comet

In that case, the kernel will only receive the balance, but this huge amount will be passed along to the respective internal deposit function (e.g. **_stDepositAssets**), increasing the stored balance to the max. This also affects the raw NAV calculation later on,

A similar case happens for fee-on-transfer tokens. These will take some fee amount from the transfer amount, resulting in less value being transferred to the kernel, but the full amount will still be accounted for when increasing the stored balance and later on when calculating the raw NAV.

```
    function deposit(TRANCHE_UNIT _assets, address _receiver) public virtual override whenNotPaused
restricted returns (uint256 shares) {
        require(_receiver != address(0), ERC20InvalidReceiver(address(0)));
        require(_assets != toTrancheUnits(0), MUST_DEPOSIT_NON_ZERO_ASSETS());

        // Transfer the assets to the kernel
        IERC20(ASSET).safeTransferFrom(msg.sender, KERNEL, toUint256(_assets));

        // Deposit the assets into the Royco market and get the fraction of total assets allocated
        (NAV_UNIT valueAllocated, NAV_UNIT effectiveNAVToMintAt) =
            (TRANCHE_TYPE() == TrancheType.SENIOR ? IRoycoKernel(KERNEL).stDeposit(_assets) :
IRoycoKernel(KERNEL).jtDeposit(_assets));

        // effectiveNAVToMint at can be zero initially when the tranche is deployed
        require(valueAllocated != ZERO_NAV_UNITS, INVALID_VALUE_ALLOCATED());

        // valueAllocated represents the value of the assets deposited in the asset that the tranche's NAV is
denominated in
        // shares are minted to the user at the effective NAV of the tranche
        // effectiveNAVToMintAt is the effective NAV of the tranche before the deposit is made, ie. the NAV at
which the shares will be minted
        shares = _convertToShares(valueAllocated, totalSupply(), effectiveNAVToMintAt, Math.Rounding.Floor);

        // Mint the shares to the receiver
        _mint(_receiver, shares);

        emit Deposit(msg.sender, _receiver, _assets, shares);
    }
```

Remediation:

We recommend using a pre- and post-balance check to determine how much value was actually transferred during the **safeTransferFrom** call.

If this is not preferred, we recommend cautiously onboarding new tokens into the protocol. While fee-on-transfer tokens are usually obvious, lesser known quirks such as infinite-transfer for Compound tokens could still cause potential trouble.

Commentary from the client:

"The protocol won't support Fee-on-Transfer token."

ROYCO3-1 | Redundant blacklist check in balance update hook

Fixed

Severity:	Probability:	Impact:
Informational	Likely	Informational

Path:

src/kernels/base/RoycoKernel.sol:preTrancheBalanceUpdateHook#L535-L551

Description:

The function **preTrancheBalanceUpdateHook** in the kernel is called from the internal **_update** function in the tranche, which is invoked upon ERC20 transfers, mints or burns. It directly passes the **_from** and **_to**.

For a mint, the **_from** would be **address(0)** and for a burn the **_to** would be **address(0)**. The function correctly checks the burn case when checking the access manager restrictions on the deposit function to check if the receiver can indeed deposit into the tranche.

However, it still performs the blacklist checks with **isBlacklisted** in the beginning on both parameters, which would be a redundant storage read on a mint or burn.

```
function preTrancheBalanceUpdateHook(address _from, address _to, uint256 _value) external
override(IRoycoKernel) onlyTranche whenNotPaused {
    // Check if the sender or recipient are blacklisted
    require(!isBlacklisted(_from), ACCOUNT_BLACKLISTED(_from));
    require(!isBlacklisted(_to), ACCOUNT_BLACKLISTED(_to));

    // If transferring shares, ensure that the recipient is a whitelisted LP for the tranche
    // It is assumed that the sender is already a whitelisted LP
    if (_to != address(0)) {
        address authority = authority();
        // Check if the to address can call the deposit function on the tranche
        (bool isWhitelistedTrancheLP,) = IAccessManager(authority).canCall(_to, msg.sender,
IRoycoVaultTranche.deposit.selector);
        require(_to != authority && isWhitelistedTrancheLP,
ACCOUNT_NOT_WHITELISTED_TRANCHE_LP(_to));
    }

    // Call the market specific pre-balance update hook
    _preTrancheBalanceUpdate(_from, _to, _value);
}
```

Remediation:

We recommend only checking the blacklist if the value is not **address(0)**. The check for **_to** can also be merged with the access control check.

For example:

```
if (_from != address(0)) {
  require(!isBlacklisted(_from), ACCOUNT_BLACKLISTED(_from));
}
if (_to != address(0)) {
  require(!isBlacklisted(_to), ACCOUNT_BLACKLISTED(_to));

  address authority = authority();
  // Check if the to address can call the deposit function on the tranche
  (bool isWhitelistedTrancheLP,) = IAccessManager(authority).canCall(_to, msg.sender,
IRoycoVaultTranche.deposit.selector);
  require(_to != authority && isWhitelistedTrancheLP, ACCOUNT_NOT_WHITELISTED_TRANCHE_LP(_to));
}
```

ROYCO3-2 | Missing event for market state to perpetual state

Fixed

Severity:	Probability:	Impact:
Informational	Likely	Informational

Path:

src/accountant/RoycoAccountant.sol:preOpSyncTrancheAccounting#L105-L150

Description:

In the RoycoAccountant, the function **preOpSyncTrancheAccounting** will emit an event on line 142 whenever the market state has changed from perpetual to a fixed term:

```
// If the market transitioned from a perpetual to a fixed-term state, set the end timestamp of the fixed-term
if (initialMarketState == MarketState.PERPETUAL && state.marketState == MarketState.FIXED_TERM) {
    $.fixedTermEndTimestamp = state.fixedTermEndTimestamp;
    emit FixedTermCommenced(state.fixedTermEndTimestamp);
}
```

However, when it goes back from a fixed term to a perpetual state, which can happen when either the timestamp has passed or when the impermanent loss has been cleared, it does not emit an event and silently goes back to the perpetual state.

This can make off-chain tracking more cumbersome.

```
    function preOpSyncTrancheAccounting(
        NAV_UNIT _stRawNAV,
        NAV_UNIT _jtRawNAV
    )
        public
        override(IRoycoAccountant)
        onlyRoycoKernel
        returns (SyncedAccountingState memory state)
    {
        // Get the storage pointer to the accountant state
        RoycoAccountantState storage $ = _getRoycoAccountantStorage();

        // Preview synchronization of the tranche NAVs and impermanent losses
        MarketState initialMarketState;
        bool yieldDistributed;
        NAV_UNIT jtImpermanentLossErased;
```

```
        (state, initialMarketState, yieldDistributed, jtImpermanentLossErased) =
_previewSyncTrancheAccounting(_stRawNAV, _jtRawNAV, _accrueJTYieldShare());

        // ST yield was split between ST and JT
        if (yieldDistributed) {
            // Reset the accumulator and update the last yield distribution timestamp
            delete $.twJTYieldShareAccruedWAD;
            $.lastDistributionTimestamp = uint32(block.timestamp);
        }

        // Checkpoint the resulting market state, mark-to-market NAVs, and impermanent losses
        $.lastMarketState = state.marketState;
        $.lastSTRawNAV = _stRawNAV;
        $.lastJTRawNAV = _jtRawNAV;
        $.lastSTEffectiveNAV = state.stEffectiveNAV;
        $.lastJTEffectiveNAV = state.jtEffectiveNAV;
        $.lastSTImpermanentLoss = state.stImpermanentLoss;
        $.lastJTImpermanentLoss = state.jtImpermanentLoss;

        // If the market transitioned from a perpetual to a fixed-term state, set the end timestamp of the fixed-term
        if (initialMarketState == MarketState.PERPETUAL && state.marketState == MarketState.FIXED_TERM) {
            $.fixedTermEndTimestamp = state.fixedTermEndTimestamp;
            emit FixedTermCommenced(state.fixedTermEndTimestamp);
        }

        // If the JT Coverage IL was erased, signal the resetting
        if (jtImpermanentLossErased != ZERO_NAV_UNITS) {
            emit JTImpermanentLossReset(jtImpermanentLossErased);
        }

        emit TrancheAccountingSynced(state);
    }
```

Remediation:

We recommend to consider also emitting an event when the market state flips back from fixed term to perpetual.

ROYCO3-7 | Redundant variable jtClaimOnSelfRawNAV in _applySeniorTrancheSelfLiquidationBonus

Fixed

Severity:	Probability:	Impact:
Informational	Rare	Informational

Path:

src/kernels/base/RoycoKernel.sol#L827

Description:

In **RoycoKernel._applySeniorTrancheSelfLiquidationBonus()**, the variable **jtClaimOnSelfRawNAV** is returned by **_decomposeNAVClaims(_state)** function on line 827. However, it is never used, making it unnecessary.

```
(,, NAV_UNIT jtClaimOnSTRawNAV, NAV_UNIT jtClaimOnSelfRawNAV) = _decomposeNAVClaims(_state);
```

Remediation:

Consider removing this variable if it serves no purpose.

```
-- (,, NAV_UNIT jtClaimOnSTRawNAV, NAV_UNIT jtClaimOnSelfRawNAV) = _decomposeNAVClaims(_state);
++ (,, NAV_UNIT jtClaimOnSTRawNAV,) = _decomposeNAVClaims(_state);
```

ROYCO3-9 | Redundant capping of jtYieldShareWAD in _computeCurrentJtYieldShare

Acknowledged

Severity:	Probability:	Impact:
Informational	Rare	Informational

Path:

src/ydm/AdaptiveCurveYDM_V1.sol#L275
src/ydm/AdaptiveCurveYDM_V2.sol#L263

Description:

At the end of the function **AdaptiveCurveYDM_V1._computeCurrentJtYieldShare()**, the output **jtYieldShareWAD** is capped at **WAD** if it exceeds this value.

```
function _computeCurrentJtYieldShare(
    uint256 _steepnessWAD,
    int256 _normalizedDeltaFromTargetWAD,
    uint256 _jtYieldShareAtTargetWAD
)
    internal
    pure
    returns (uint256 jtYieldShareWAD)
{
    ...

    if (jtYieldShareWAD > WAD) jtYieldShareWAD = WAD;
}
```

However, **jtYieldShareWAD** is only used in two places within the **RoycoAccountant** contract, and in both cases the value is already capped at **WAD** after being returned.

```
function _previewJTYieldShareAccrual() internal view returns (uint192) {
    ...

    uint256 jtYieldShareWAD =
        IYDM($.ydm).previewJTYieldShare($.lastMarketState, $.lastSTRawNAV, $.lastJTRawNAV, $.betaWAD,
$.coverageWAD, $.lastJTEffectiveNAV);

    if (jtYieldShareWAD > WAD) jtYieldShareWAD = WAD;
}
```

```
function _accrueJTYieldShare() internal returns (uint192 twJTYieldShareAccruedWAD) {
    uint256 jtYieldShareWAD =
        IYDM($.ydm).jtYieldShare($.lastMarketState, $.lastSTRawNAV, $.lastJTRawNAV, $.betaWAD,
$.coverageWAD, $.lastJTEffectiveNAV);

    if (jtYieldShareWAD > WAD) jtYieldShareWAD = WAD;
}
```

As a result, the capping of **jtYieldShareWAD** in **AdaptiveCurveYDM_V1._computeCurrentJtYieldShare()** is redundant. The same applies to the **WAD** capping in **AdaptiveCurveYDM_V2._jtYieldShare()**.

Remediation:

Consider removing the redundant **WAD** capping in **AdaptiveCurveYDM_V1._computeCurrentJtYieldShare()** and **AdaptiveCurveYDM_V2._jtYieldShare()**, since the returned value is already capped by the caller in **RoycoAccountant**. This reduces duplicate logic and simplifies the codebase while maintaining the same safety guarantees.

Commentary from the client:

"Leaving as it is for a safety check."

hexens x ROYCO

